

SparkInflux

An InfluxDB tag-history provider for Ignition, covering both 8.1.19+ (TagHistoryProvider) and 8.3.2+ (AbstractHistorian), with pipelined writes, disk store-and-forward, and streaming reads.

Version 0.2.0 Ignition 8.1.19+ and 8.3.2+ Runs on: any gateway

1. [Overview](#)
2. [What You Need](#)
3. [Install](#)
4. [License](#)
5. [Configure](#)
6. [Operate](#)
7. [Verify](#)
8. [Troubleshoot](#)
9. [Reference](#)
10. [Support](#)

1. Overview

SparkInflux is a tag-history provider that writes Ignition tag history to InfluxDB (1.8, 2.x, or 3.x) instead of Ignition's default SQL history. One codebase covers Ignition 8.1.19+ via TagHistoryProvider and 8.3.2+ via AbstractHistorian (sibling repo sparkinflux-83). Writes are pipelined and gzip-compressed by default, buffered to a disk ring during an InfluxDB outage instead of being dropped, and reads stream row-by-row with server-side aggregate pushdown instead of materializing the full result set in memory.

A gateway can run several providers at once, each with its own connection, write pipeline, and store-and-forward ring. SparkInflux is a standalone paid product, never bundled into SparkBridge itself.

2. What You Need

- Ignition 8.1.19+ or 8.3.2+.
- An InfluxDB server: version 1.8, 2.x, or 3.x.
- No other SparkBridge module is required. SparkInflux is standalone with its own license.

3. Install

1. Install the signed SparkInflux module through Config > Modules > Install or Upgrade a Module; no build step is required for the release bundle.
2. Signed release builds need a real certificate; a development build produces an unsigned .modl that prompts the same untrusted-certificate warning on install.

4. License

Runs on Ignition's 2-hour module trial until licensed, same as every SparkBridge-suite module. Drop a sparkbridge-license.key in the gateway's data directory to lift the pause within 30 seconds; this clears the license label on status.json. Standalone paid product; never bundled into SparkBridge itself.

5. Configure

1. Config > SparkInflux > Providers > Add. Fill in Name, InfluxDB Version (V1/V2/V3), URL, Token, Org, Bucket, then Save. The row starts its own connection, write pipeline, and store-and-forward ring immediately, no gateway restart.
2. In Designer, open a tag's History properties, set History Provider to the provider name just created, and set a sample or scan-class rate. There is no headless gateway-web path to create or edit tag history; this step needs Designer.

Category Fields

Connection Name, Enabled, Version (V1/V2/V3), Url, Token, Org, Bucket

Write Measurement, Gzip, Concurrent Write Requests, Batch Size, Flush Interval, Store-and-Forward Cap, Deadband Enabled/Absolute/Percent/Min Interval/Overrides, Adaptive Batch Enabled/Target Low/Target High/Min/Max

Read Max Query Rows, Browse Cache TTL, Browse Window

Routing Routing Rules (prefix=bucket[:retentionSeconds], one per line), Downsample Rules

Advanced Request Timeout

Changing a row restarts only that provider; every other provider on the same gateway keeps its ring buffer and connections untouched. There is no test-connection button on the edit form; use testcreate.json or the status routes below instead.

Deadband / change-only storage

Off by default. Per tag path, a numeric value is stored if the absolute move clears Deadband Absolute, the relative move clears Deadband Percent, or Deadband Min Interval has elapsed since the last stored point. Boolean and String values are always change-only. A quality change is always stored regardless of value movement. Deadband Overrides layers per-path rules on top of the provider defaults: path=abs:0.5,pct:1,min:1000;path2=abs:2. Scripted writes via `system.influx.write()` deliberately bypass deadband.

Adaptive batch sizing

Off by default (fixed Batch Size otherwise). Write latency and any server Retry-After adjust the effective batch-flush threshold within the configured min/max bounds (default 500 to 50000 lines): low latency grows the batch, high latency or throttling shrinks it, adjusted at most once per 500 ms.

Scripting

```
system.influx.providers()
system.influx.write("influx", None, {"tagPath": "Line1/Temp"}, {"fval": 72.4})
rows = system.influx.query("influx", 'from(bucket:"ignition") |> range(start:-1h) |> filter(fn:(r) => r.tagPath == "Line1/Temp")')
system.influx.status("influx")
```

Available from gateway-scope scripts: scan class, tag change script, or gateway event script.

Multi-bucket routing and config-as-code

A provider's Routing Rules field routes a tag path to the first matching prefix rule, else the provider's own Bucket. GET /data/com.sparkbridge.sparkinflux/config.json[?includeSecrets=1] returns every provider row as JSON (tokens redacted unless loopback and includeSecrets=1); POST to the same path applies the same shape idempotently by name, restarting only changed providers. Omitting token on an update leaves the existing credential untouched.

6. Operate

Day to day, watch status.json and providers.json for read/write lane health per provider. On an InfluxDB outage, writes buffer to a disk ring instead of being dropped, and drain back out on recovery; a full ring is quarantined rather than silently deleted. A malformed write (a 400 partial-write response) quarantines only the offending lines, not the whole batch. Reads use a per-provider in-memory cache for repeated aggregate queries (default 5 s TTL, capped at 50,000 rows and a configurable byte budget), invalidated the moment a write flush touches the same tag paths.

Downsampling is managed as scheduled Flux tasks via POST /data/com.sparkbridge.sparkinflux/downsample.json on v1.8/v2 providers; v3 providers return a clear unsupported-operation error since InfluxDB 3.x has no Tasks API yet.

7. Verify

- 1. GET /data/com.sparkbridge.sparkinflux/selftest.json?provider=<name>&points=300 pushes real points through history storage and reads them back, returning pointsSent, pointsWritten, rowsRead, and ok.
- 2. GET /data/com.sparkbridge.sparkinflux/status.json for product/version/license state, and GET /data/com.sparkbridge.sparkinflux/providers.json for per-provider read/write lane state.
- 3. GET /data/com.sparkbridge.sparkinflux/schema.json?provider=<name> lists measurements, tag keys, series-cardinality estimates, and warnings (cardinality over 100,000, un-migrated Kymera-style [provider] prefixes, a measurement name mismatch).

All status/config routes are loopback-only unless sparkinflux.rest.readToken is set in the gateway's secrets file, in which case they accept Authorization: Bearer <token>.

8. Troubleshoot

Symptom	Cause	Fix
No test-connection button on the provider edit form	By design; not implemented as a Wicket AJAX component	Save the row, then check status.json / providers.json, or run testcreate.json
Tag history not appearing under a new provider	Tag's History Provider not set, or scan/sample rate not configured	In Designer, set the tag's History Provider to the new provider name and set a rate; there is no gateway-web path for this
Points stop writing during an Influx outage	Expected; writes buffer to the disk ring instead of being sent	No action needed; the ring drains automatically on recovery unless it hits its cap, at which point excess is quarantined, not deleted
One bad line fails an entire write batch	Should not happen; a 400 partial-write response quarantines only the offending lines	Check providers.json for the quarantine count and inspect the quarantined lines
status.json / providers.json return nothing over the network	Routes are loopback-only by default	Set sparkinflux.rest.readToken in the gateway secrets file and pass Authorization: Bearer <token>
Downsample request fails on a V3 provider	InfluxDB 3.x has no Tasks API	Use downsampling only on v1.8/v2 providers, or downsample externally for v3
Aggregate reads look stale immediately after a write	Should not happen; cache is invalidated on write flush for touched tag paths	Confirm the write actually flushed (check batch/flush interval settings); otherwise report as a bug
Config.json includeSecrets returns redacted tokens	Not on loopback, even with a valid read token	Call includeSecrets=1 from the gateway host itself; a read token only grants health polling, not credential export

9. Reference

Item	Value
REST base	/data/com.sparkbridge.sparkinflux
Selftest	GET .../selftest.json?provider=<name>&points=300
Status	GET .../status.json, GET .../providers.json
Schema inspector	GET .../schema.json?provider=<name>
Test-and-create	GET .../testcreate.json?provider=<name>[&createBucket=1&retentionSeconds=N]
Config-as-code	GET/POST .../config.json[?includeSecrets=1]
Downsampling	POST .../downsample.json?provider=&source=&target=&every=&fn=[&name=] (v1.8/v2 only)
Read token	Gateway secret sparkinflux.rest.readToken, else loopback-only
Adaptive batch bounds	Default min 500, max 50,000 lines; adjusts at most once per 500 ms
Read cache	Default 5 s TTL, 50,000-row cap, configurable byte budget, LRU-evicted
Sibling repo	sparkinflux-83 for the Ignition 8.3.2+ AbstractHistorian line
License key path	<data>/sparkbridge-license.key, re-read within 30 s

10. Support

[SparkInflux install and config reference](#) · [Contact](#) · [Downloads and release notes](#)